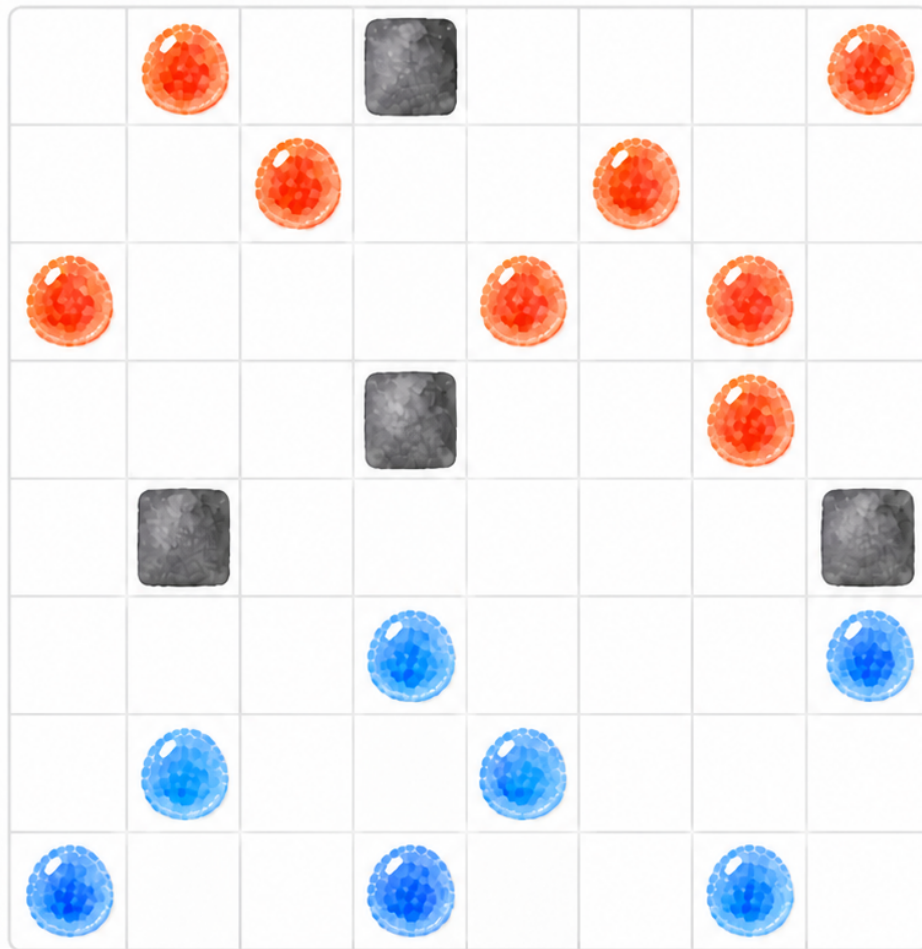


Projet Blob War

Mohammed Akram Lrhorfi : équipe 17 : Dogue Dieyeneba

Mai 2026



Résumé

Blob War est un jeu combinatoire a information parfaite sur un plateau 8×8 . A chaque tour, un joueur duplique ou deplace un pion, puis convertit les voisins adverses. Comme l'arbre de jeu croit rapidement, une recherche exhaustive est impossible dans le temps du tournoi. Nous comparons donc une strategie gloutonne, un Minimax a profondeur fixe, une version Alpha-Beta parallele intermediaire et une version Alpha-Beta optimisee, en mesurant le temps par coup, la profondeur atteinte, les noeuds explores, les coupures Alpha-Beta et le taux de victoire.

1 Modelisation du jeu

1.1 Etat du plateau

Dans le code fourni, le plateau est represente par deux tableaux bidimensionnels :

- `bidiarray<Sint16> _blobs` indique l'occupant de chaque case : `-1` pour une case vide, sinon l'identifiant du joueur ;
- `bidiarray<bool> _holes` indique les cases interdites ;
- `_current_player` stocke le joueur qui doit jouer.

Un coup est represente par la structure `movement`, qui contient les coordonnees d'origine et de destination .

1.2 Application d'un coup

La fonction `applyMove()` simule les regles de Blob War. Elle distingue les deux types de déplacements par la distance de Chebyshev :

- si la distance vaut 1, le pion est duplique ;
- si la distance vaut 2, la case source est videe ;
- la destination devient la propriete du joueur courant ;
- les voisins adverses de la destination sont convertis.

Dans les strategies, nous supposons que les coups donnees a `applyMove()` sont deja valides, car ils proviennent de `computeValidMoves()`.

1.3 Generation des coups

La fonction `computeValidMoves()` parcourt tous les pions du joueur courant. Pour chaque pion, elle examine les cases dans un carre de rayon 2. Une case destination est acceptee si elle est dans le plateau, non trouee, vide, et differente de la case source.

Si b designe le nombre moyen de coups legaux, alors les algorithmes de recherche parcourent un arbre de facteur de branchement approximatif b . A profondeur d , une recherche Minimax brute demande donc de l'ordre de $O(b^d)$ noeuds.

2 Strategies de recherche

2.1 Strategie gloutonne

La strategie gloutonne est implementee dans `strategy_glouton.cc`. Elle constitue notre base-line. Pour chaque coup legal, elle simule le coup, evalue le plateau obtenu, puis conserve le coup qui maximise le score.

```
best_score = -INF
for mv in valid_moves:
    state = current_state
    state.applyMove(mv)
    score = state.estimateCurrentScore()
    if score > best_score:
        best_score = score
        best_move = mv
```

Cette strategie est tres rapide, car elle ne regarde qu'un coup en avant. Sa limite est qu'elle ne tient pas compte des reponses adverses. Elle peut donc choisir un coup localement bon mais dangereux au tour suivant.

2.2 Minimax sequentiel

Le Minimax est implemente dans `strategy_minimax.cc`. Il explore l'arbre de jeu jusqu'a une profondeur fixe, actuellement $d = 3$. Les noeuds du joueur courant maximisent le score, tandis que les noeuds adverses le minimisent.

```
minimax(state, depth, maximizing):
    if depth == 0:
        return evaluate(state)

    moves = validMoves(state)
    if moves.empty():
        return minimax(passTurn(state), depth - 1, !maximizing)

    if maximizing:
        return max(minimax(apply(mv), depth - 1, false))
    else:
        return min(minimax(apply(mv), depth - 1, true))
```

Le cas ou un joueur ne peut pas jouer est traite explicitement : on passe au joueur suivant. Si aucun des deux joueurs ne peut jouer, l'etat est evalue directement.

La complexite theorique est :

$$W_{minimax}(d) = O(b^d).$$

Cette strategie anticipe mieux que le glouton, mais elle explore beaucoup de noeuds inutiles. Elle sert donc de reference pour mesurer l'apport de l'elagage Alpha-Beta.

2.3 Alpha-Beta parallele intermediaire

La version intermediaire est implementee dans `strategy_alphabeta0.cc`. Elle ajoute deux idees importantes par rapport au Minimax :

- l'elagage Alpha-Beta;
- la parallelisation des coups racine avec OpenMP.

L'algorithme Alpha-Beta conserve deux bornes :

- α : meilleure valeur garantie pour le joueur maximisant ;
- β : meilleure valeur garantie pour le joueur minimisant.

Des qu'on obtient $\alpha \geq \beta$, le sous-arbre courant ne peut plus influencer la decision finale et peut etre coupe.

```
if alpha >= beta:
    return alpha // coupure beta
```

Au niveau racine, les coups sont independants. Nous utilisons donc OpenMP :

```
#pragma omp parallel for schedule(dynamic)
for (int i = 0; i < moveCount; i++) {
    Strategy child(*this);
    child.applyMove(moves[i]);
    score = child.alphaBeta(...);
}
```

Le choix `schedule(dynamic)` est important : deux sous-arbres de meme profondeur peuvent avoir des couts tres differents selon les coupures Alpha-Beta. Une distribution dynamique equilibre donc mieux le travail entre threads qu'une repartition statique.

2.4 Alpha-Beta parallele optimise

La strategie finale est implementee dans `strategy.cc`. Elle reprend Alpha-Beta et ajoute plusieurs optimisations :

- tri des coups ;
- approfondissement iteratif ;
- sauvegarde immediate d'un coup valide ;
- sauvegarde apres chaque profondeur terminee ;
- partage d'une borne α atomique entre threads ;
- exploration sequentielle d'un prefixe de coups avant parallelisation du reste.

L'approfondissement iteratif permet de respecter la contrainte de temps du tournoi. Le processus de recherche peut etre tue par le moteur apres le timeout ; il est donc essentiel d'avoir deja sauvegarde un coup valide.

```
bestMove = moves[0]
saveBestMove(bestMove)

for depth = INITIAL_SEARCH_DEPTH; ; depth++:
    depthBestMove = alphaBetaRoot(depth)
    bestMove = depthBestMove
    saveBestMove(bestMove)
```

3 Heuristique d'evaluation

3.1 Heuristique materielle simple

Les premieres strategies utilisent une evaluation simple :

$$Material = pions_{moi} - pions_{adversaire}.$$

Elle est pertinente en fin de partie, car le gagnant est determine par le nombre de pions. En revanche, elle est pauvre en debut et milieu de partie : elle ne mesure ni la mobilite, ni la qualite positionnelle, ni le risque de conversion au prochain tour.

3.2 Heuristique multi-criteres

Les versions Alpha-Beta utilisent une heuristique plus riche :

$$Eval(s) = w_m \cdot Material + w_{mob} \cdot Mobility + w_p \cdot Position.$$

Le terme de mobilite est :

$$Mobility = legalMoves(moi) - legalMoves(adversaire).$$

Le terme positionnel repose sur une table de poids fixe. Les cases centrales ont un poids plus eleve, car elles offrent davantage de possibilites de deplacement et de conversion, surtout sur la map standard.

```
static const int WEIGHT[8][8] = {
    {2, 3, 4, 4, 4, 4, 3, 2},
    {3, 4, 5, 6, 6, 5, 4, 3},
    {4, 5, 7, 8, 8, 7, 5, 4},
    {4, 6, 8, 10, 10, 8, 6, 4},
    {4, 6, 8, 10, 10, 8, 6, 4},
    {4, 5, 7, 8, 8, 7, 5, 4},
```

```
{3, 4, 5, 6, 6, 5, 4, 3},  
{2, 3, 4, 4, 4, 4, 3, 2}  
};
```

3.3 Ponderation selon la phase de jeu

L'heuristique s'adapte selon le nombre de cases vides :

```
if (empty * 5 < playable) {  
    return 220 * material + mobility;  
}  
  
if (empty * 2 > playable) {  
    return 115 * material + 3 * mobility + 4 * positionBalance;  
}  
  
return 145 * material + 2 * mobility + 3 * positionBalance;
```

En debut de partie, la mobilite et la position sont importantes. En fin de partie, le materiel devient dominant, car le score final depend directement du nombre de pions.

3.4 Tri des coups

Le tri des coups est une optimisation centrale pour Alpha-Beta. En effet, Alpha-Beta ne change pas le resultat du Minimax, mais son efficacite depend fortement de l'ordre d'exploration. Avec un mauvais ordre, le cout reste proche de :

$$O(b^d).$$

Avec un ordre presque optimal, Alpha-Beta peut approcher :

$$O(b^{d/2}).$$

Dans notre code, `moveOrderingScore()` favorise :

- les duplications ;
- les coups qui convertissent beaucoup de pions adverses ;
- les destinations ayant un bon poids positionnel.

4 Parallelisation

4.1 Travail et profondeur

Dans les algorithmes de recherche, on distingue deux mesures de complexite :

- le travail W , c'est-a-dire le cout total sur un processeur ;
- la profondeur D , c'est-a-dire la longueur du chemin critique.

Sur p processeurs, la loi de Brent donne l'intuition suivante :

$$T_p \approx \frac{W}{p} + D.$$

Dans Blob War, le Minimax pur a un travail tres eleve, de l'ordre de $O(b^d)$. Le but d'Alpha-Beta est de diminuer le travail effectif W en coupant des sous-arbres. Le but de la parallelisation est ensuite de repartir le travail restant entre plusieurs coeurs.

4.2 Pourquoi Alpha-Beta est difficile a paralleliser

Alpha-Beta contient une dependance sequentielle : les bornes α et β sont mises a jour apres chaque coup explore. Une boucle parallele naive peut donc explorer trop de sous-arbres avant d'avoir une bonne borne.

```
for mv in moves:
    score = search(mv, alpha, beta)
    alpha = max(alpha, score)
    if alpha >= beta:
        break
```

Le **break** montre que l'algorithme n'est pas naturellement un **for**// parfait. Il faut paralleliser de facon speculative mais controlee.

4.3 Strategie cascade

Dans la strategie finale, nous explorons d'abord un prefixe des meilleurs coups sequentiellement. La taille de ce prefixe est :

$$k = \max(1, \lfloor \sqrt{b} \rfloor).$$

Cette premiere phase donne une meilleure valeur de α . Ensuite, les autres coups sont explores en parallele avec une borne initiale plus forte.

```
sequentialCount = sqrt(moveCount)

for i in 0..sequentialCount-1:
    search moves[i] sequentially
    update alpha

#pragma omp parallel for schedule(dynamic)
for i in sequentialCount..moveCount-1:
    search moves[i] in parallel
```

Cette idee est proche des algorithmes en cascade : on traite une petite partie du probleme avec une methode plus sequentielle pour reduire le travail inutile, puis on exploite le parallelisme sur le reste.

L'objectif de ce prefixe sequentiel est proche d'un *killer move* : obtenir rapidement une bonne borne pour provoquer plus de coupures et reduire le travail des autres threads.

5 Instrumentation et protocole experimental

5.1 Traces d'execution

Chaque strategie affiche une ligne de trace au format suivant :

```
BW_TRACE algo=alphabeta_final depth=5 nodes=123456 cutoffs=7890
```

Ces traces permettent de mesurer :

- la profondeur atteinte ;
- le nombre de noeuds explores ;
- le nombre de coupures Alpha-Beta ;
- le temps moyen par coup ;

Les experiences sont effectuees sur la carte **standard**, avec un timeout de 2 secondes par coup. Chaque algorithme joue contre chaque autre, dans les deux ordres : une partie ou le premier algorithme commence, puis une partie ou le second commence.

```
python3 benchmark_blobwar.py --timeout 2 --games-per-pair 1
```

6 Resultats experimentaux

6.1 Temps moyen par coup

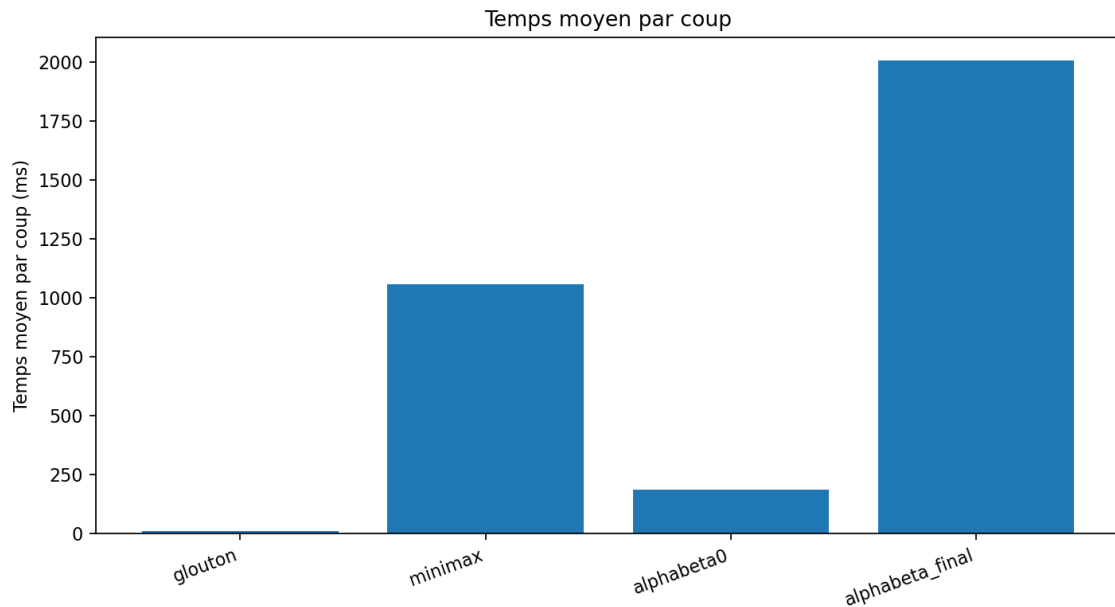
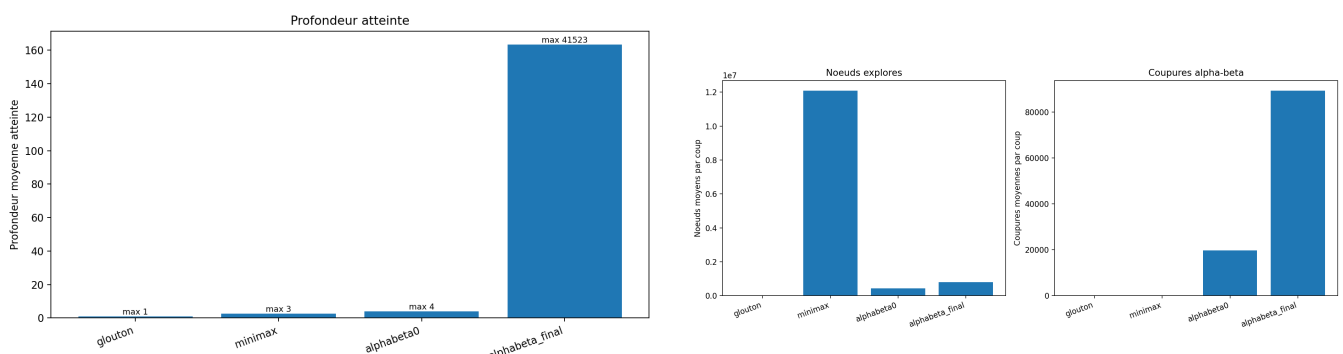


FIGURE 1 – Temps moyen par coup pour chaque strategie.

Le glouton est attendu comme la strategie la plus rapide, car il n’explore qu’un niveau. Minimax est plus couteux, car il explore l’arbre sans elagage. Les versions Alpha-Beta utilisent davantage le temps disponible pour chercher plus profond, mais evitent une partie du travail grace aux coupures.

6.2 Profondeur atteinte et noeuds explores



(a) Profondeur moyenne et maximale atteinte.

(b) Nombre moyen de noeuds explores et nombre moyen de coupures Alpha-Beta.

FIGURE 2 – Profondeur atteinte (gauche) et travail exploratoire en noeuds/coupures (droite).

Les strategies a profondeur fixe, comme le Minimax et **alphabeta0**, ont une profondeur stable. La version finale utilise l’approfondissement iteratif : la profondeur atteinte depend donc de la position courante et du temps disponible. Le nombre de noeuds explores ne doit pas etre interprete

seul : une strategie peut explorer moins de noeuds mais obtenir de meilleurs resultats si le tri des coups place les bons coups en premier et provoque davantage de coupures.

6.3 Taux de victoire

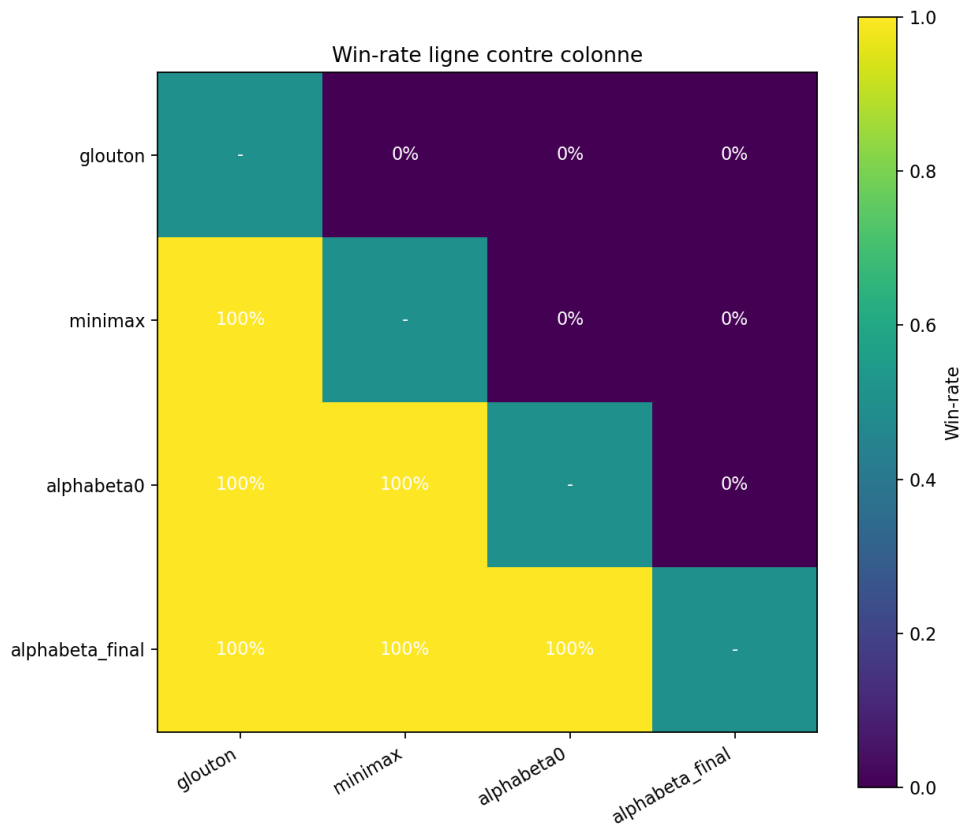


FIGURE 3 – Win-rate ligne contre colonne.

Le taux de victoire est la mesure finale de qualite en situation de jeu. Nous testons les deux ordres de depart, car une strategie peut se comporter differemment en tant que joueur 1 ou joueur 2.

Globalement, le glouton est une baseline rapide mais myope : il peut choisir un gain immediat qui ouvre une reponse adverse forte. Minimax corrige en partie ce defaut par l'anticipation, mais son cout exponentiel devient vite limitant, meme a profondeur 3. Alpha-Beta garde la meme logique de decision que Minimax a profondeur egale, tout en reduisant le travail grace aux coupures ; son efficacite depend fortement du tri des coups. La version finale ajoute enfin un parallelisme controle au niveau racine : elle explore d'abord quelques bons coups pour obtenir une borne α utile, puis parallelise le reste, ce qui evite une partie du surtravail d'un Alpha-Beta parallele trop naif.

7 Conclusion

Ce projet nous a permis de construire progressivement une intelligence artificielle pour Blob War. La progression glouton, Minimax, Alpha-Beta puis Alpha-Beta parallele montre que la performance vient surtout de la combinaison entre une bonne heuristique, un tri efficace des coups, des coupures nombreuses et une gestion correcte du timeout par approfondissement iteratif. Des pistes restent ouvertes, notamment les bitboards 64 bits, les killer moves, Monte-Carlo/MCTS pour guider le tri et un work-stealing plus fin.